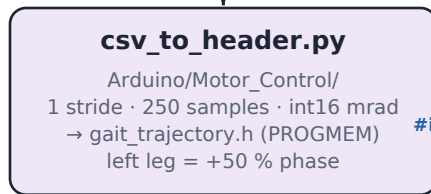
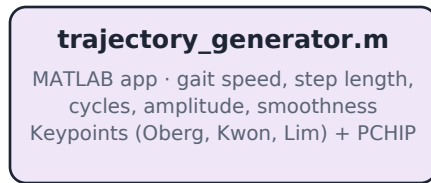


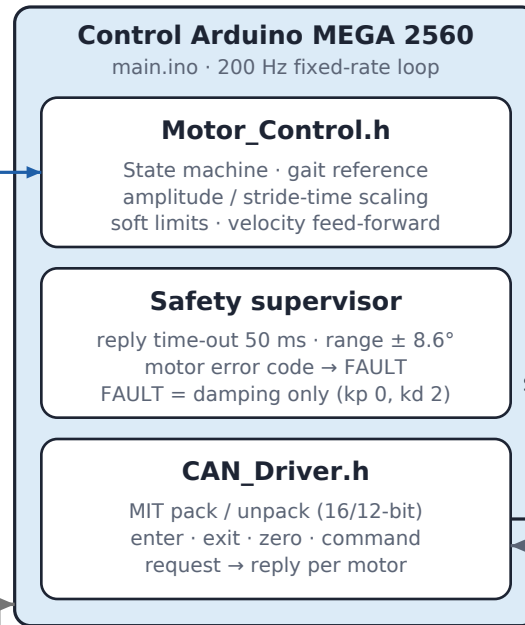
OpenRehEx — System Architecture

Open-source 6-DoF lower-limb rehabilitation exoskeleton · end-to-end data flow from trajectory design to actuation and monitoring

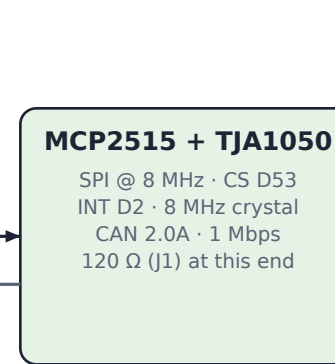
1 TRAJECTORY GENERATION (offline)



2 CONTROL (embedded, real time)



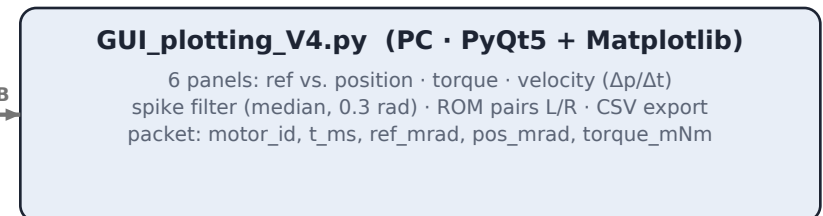
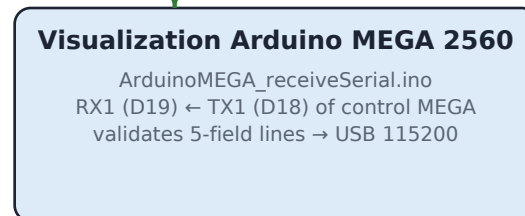
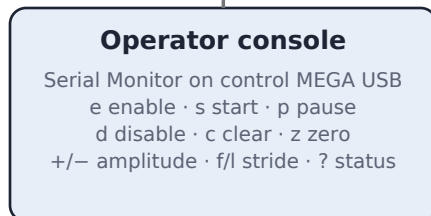
3 CAN INTERFACE



4 ACTUATION



5 VISUALIZATION (optional — the exoskeleton runs without it)

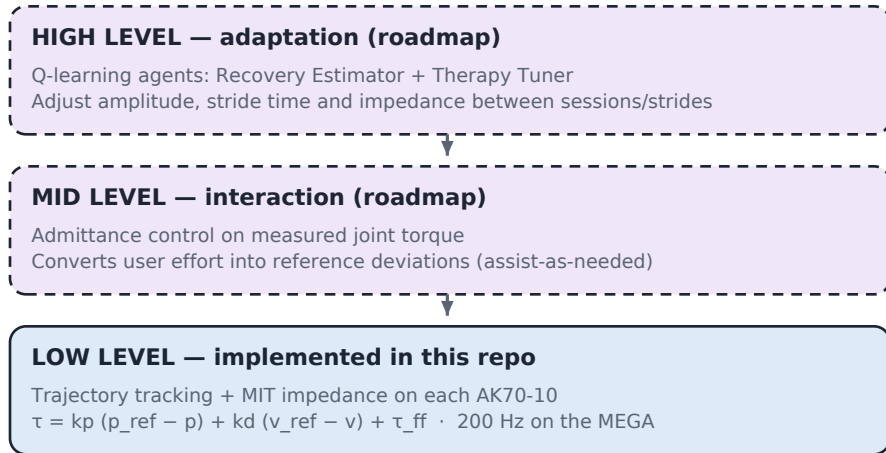


* Vendor-listed AK70-10 figures; confirm against the datasheet of your units. Colors: yellow CAN_H · blue CAN_L · green UART · gray USB.

Control Architecture & Firmware

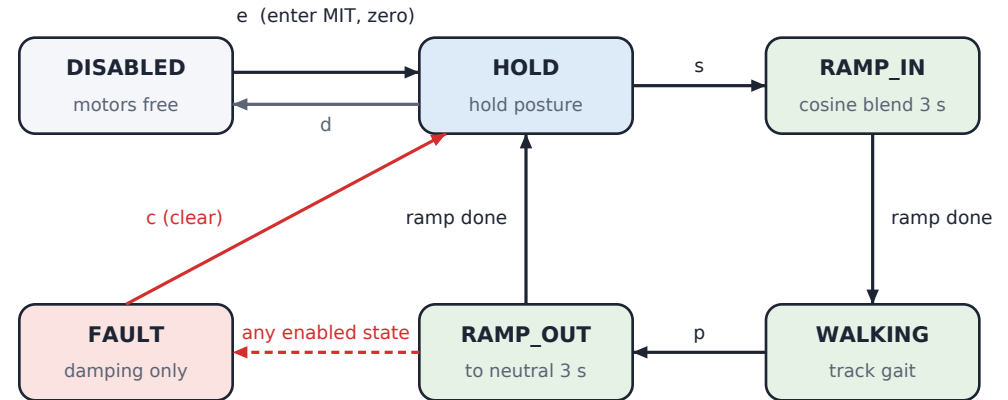
Control hierarchy, operating states and the real-time cycle of the control MEGA

Control hierarchy



Dashed = planned research layers (not in the firmware yet). Their outputs map onto `exo.setAmplitude()`, `exo.setStrideTime()`, `jointConfig kp/kd` and the `$\tau_{ff}$` slot in `buildCommand()`.

Operating states (Motor_Control.h)



FAULT triggers: no reply from a motor for $> 50 \text{ ms}$ · measured angle outside the soft limits by more than 0.15 rad · non-zero motor error code (firmware $\geq V2$). In FAULT the motors stay in MIT mode with $k_p = 0$, $k_d = 2 \text{ N}\cdot\text{m}\cdot\text{s}/\text{rad}$: the limbs are slowed, not held. Only the hardware E-stop removes motor power.

Real-time cycle (CONTROL_HZ = 200 → 5 ms)



ref = compute references (gait table interpolation, amplitude slew, ramps, soft limits) · **M1...M6** = send MIT frame to motor *i*, then wait $\leq 1.5 \text{ ms}$ for its reply

sup = safety supervisor · **tlm** = one telemetry line on Serial1.

Per-motor request/reply is required because the MCP2515 has only two RX buffers. Durations shown are estimates for a 16 MHz MEGA; check them with the '?' command (max exec μs , overruns).

CAN load $\approx 12 \text{ frames} \times \sim 0.13 \text{ ms} = 1.6 \text{ ms}$ per cycle ($\approx 30 \%$ of the bus at 200 Hz). Telemetry: $200 \text{ lines/s} \times \sim 25 \text{ B} \approx 5 \text{ kB/s}$ ($\approx 45 \%$ of 115200 baud) → each motor is plotted at $\approx 33 \text{ Hz}$.

Reference generation

$\text{phase } \phi \leftarrow \phi + \Delta t / T_{\text{stride}} \pmod{1}$ $q_{ref,R(j)} = A \cdot g_j(\phi)$ $q_{ref,L(j)} = A \cdot g_j(\phi + 0.5)$

g_j = PROGMEM table of joint *j* (hip, knee, ankle), 250 samples, linear interpolation. $A \in [0, 1]$ slews at $0.1 / \text{s}$; $T_{\text{stride}} \in [0.8, 10] \text{ s}$ (default 3 s, $A = 0.5$).

motor = direction · joint + offset $v_{ref} = \Delta q_{ref} / \Delta t$ (clamped $\pm 6 \text{ rad/s}$) $\tau_{ff} = 0$ (slot for gravity / interaction compensation)

Repository Map & Interfaces

Where each part of the system lives and the data contracts between them

Repository layout

Path	Content
CAD/Hip, Knee, Ankle, Assembly	SolidWorks parts/assemblies (OpehRehEx_V1.SLDASM, AK70-10.SLDPRT)
MATLAB/trajectory_generator.m	App: biomechanical gait trajectories → .csv / .mat / .txt
MATLAB/gait_parameters.m, examples/	Reference gait tables · slow / normal / fast demos
Arduino/main.ino	Control MEGA firmware: joint table, 200 Hz loop, console, telemetry
Arduino/CAN_Driver/CAN_Driver.h	AK70-10 MIT protocol over MCP2515 (header-only)
Arduino/Motor_Control/Motor_Control.h	State machine, gait reference, safety supervisor (header-only)
Arduino/Motor_Control/gait_trajectory.h	Auto-generated PROGMEM gait table (1 stride, mrad)
Arduino/Motor_Control/csv_to_header.py	MATLAB CSV → gait_trajectory.h converter
Arduino/Graphs_visualization/	Relay MEGA sketch + GUI_plotting_V4.py + requirements.txt
CAN_Protocol/motor_commands.md	Frame layouts, scaling, IDs, sequences, troubleshooting
CAN_Protocol/CubeMarstool_V1.32.exe	Vendor tool: IDs, firmware, MIT limits, calibration
docs/	system_architecture.pdf, wiring_diagram.pdf, src/build_docs.py

Data interfaces

Link	Physical	Content	Rate
MATLAB → converter	CSV	Time_s, Hip, Knee, Ankle (deg or rad, N strides)	offline
Converter → firmware	C header	int16 mrad ×3 × 250, GAIT_STRIDE_TIME_S	compile time
Control MEGA ↔ MCP2515	SPI	mcp_can, CS D53, INT D2	≤ 8 MHz
MCP2515 ↔ AK70-10 ×6	CAN 2.0A	MIT command 8 B / reply 6–8 B, IDs 0x01–0x06	1 Mbps
Control → Visualization MEGA	UART	id,t_ms,ref_mrad,pos_mrad,tor_mNm\n	115200 baud
Visualization MEGA → PC	USB CDC	same 5-field lines, validated	115200 baud
PC → Control MEGA	USB CDC	single-char console commands	115200 baud

Conventions

Joint angles (MATLAB, firmware, GUI): hip + flexion / – extension · knee + flexion · ankle + dorsiflexion / – plantarflexion.

Mechanical range (deg): hip [–20, 40] · knee [0, 50] · ankle [–20, 20]; used as firmware soft limits.

Motor IDs 1–3 right leg (hip, knee, ankle), 4–6 left leg; same order in main.ino JOINTS[] and GUI MOTOR_META.

Units on the wire: rad, rad/s, N·m (CAN, float-scaled); mrad and mN·m integers (UART telemetry).

Before powering with a person in the exoskeleton

- 1) Confirm MIT scaling limits in CubeMarsTool and select them in setup().
- 2) Verify each joint's 'direction' unloaded.
- 3) Enable only at neutral posture (zero_on_enable).
- 4) Test the hardware E-stop and FAULT behaviour at low amplitude.
- 5) Start with $A \leq 0.5$, stride ≥ 3 s and reduced kp.
- 6) Follow your institution's ethics / clinical-device protocol.